

Game Design Document

„Debugger3D“

Modulnummer: 4FSC0PD003/4 0326

Modulname: Games Programming

Abschluss: Games Programming Diploma

Semester: März 2026

Name: Bartosz Chelminiak

Campus: Köln

Land: Deutschland

Selbstständigkeitserklärung

Hiermit bestätige ich, dass ich die vorliegende Arbeit eigenständig erarbeitet habe und alle Hilfsmittel sowie Inhalte von Dritten vollständig angegeben wurden. Hierunter fällt zum Beispiel die korrekte Belegarbeit für die direkte oder sinngemäße Übernahme von Texten, Bild-, Audio- und Videomaterial sowie Code etc. Weiterhin die klare Kennzeichnung von Inhalten, die von anderen Personen oder technischen Hilfsmitteln wie z. B. einer künstlichen Intelligenz erstellt wurden.

Düren, den 18.08.2026

Bartosz Chelminiak

Ort, Datum

Unterschrift Student/in

Rechtevereinbarung

Hiermit räume ich dem SAE Institut das nicht exklusive, jedoch zeitlich und örtlich unbeschränkte Recht ein, die vorliegende Arbeit zum Zweck der Ausbildung sowie der Darstellung von Ausbildungsinhalten zu speichern und für Personen des SAE Instituts zugänglich zu machen.

Düren, den 18.08.2026

Bartosz Chelminiak

Ort, Datum

Unterschrift Student/in

Inhaltsverzeichnis

1 Spielkonzept.....	3
2 Entwicklungshistorie.....	3
3 Spieleablauf.....	4
4 Spielmechaniken.....	4
5.Gegner-KI.....	5
6.Level Design.....	5
7.Audio und UI.....	6
8.Änderungen im Laufe der Zeit.....	6
9.Verwendete Assets.....	7
10.Offene Punkte.....	7

1 Spielkonzept

Debugger3D ist ein First-Person-Spiel, in dem man den Albtraum eines überarbeiteten Technikers erlebt. Als besagter Techniker wird man in den Serverraumkeller einer Firma geschickt, um einige Bugs zu beseitigen. Das Problem: Der Keller ist von Bugs befallen. Nicht von Programmierfehlern, sondern von echten Krabbeltieren, Spinnen die von der Decke fallen, und Käfern, die über den Boden laufen. Kaum ist man angekommen, schließt sich hinter einem die Tür. Man hört eine Nachricht der Kundin ab und beginnt nach den Servern zu suchen. Mit und mit merkt man aber das irgendwas nicht stimmt und wird plötzlich mit richtigen Bugs konfrontiert. Ab da gibt es nur einen Weg zurück: Alle Server finden, anschalten und überleben.

Genre: First-Person Survival mit leichten Horrorelementen

Perspektive: Ego-Perspektive mit sichtbaren Armen und Waffe

Spielzeit: ca. 10 bis 15 Minuten pro Durchlauf

Engine: Unity 6

2 Entwicklungshistorie

Das Projekt startete als 2D Top-Down Spiel namens „Debugger“. Im Laufe der Entwicklung stellte sich heraus, dass Pathfinding in 2D ohne NavMesh - Unterstützung deutlich schwieriger umzusetzen war als erwartet. Da NavMesh-Baking zur Laufzeit in 3D von Haus aus funktioniert, fiel die Entscheidung, auf 3D First-Person umzustellen. Die erste Idee dieses Spiels beinhaltete außerdem mehrere Waffen, Quick Time Events und einem Bosskampf. Nach der Zwischenpräsentation wurde es aber auf ein Rundensystem umgebaut, das in einem Dungeon funktioniert. Der Grund war der Umfang: Mit wenig Elementen (Server, Schalter, Tür, Lampen) entstand ein kompletter Gameloop mit Anfang, Steigerung und Ende. Gestrichenes steht in Kapitel 8.

3 Spielablauf

Das Spiel läuft in drei Runden, die alle demselben Muster folgen:

- Server finden: Im prozedural generierten Dungeon stehen Server an zufälligen Positionen, ihre LEDs leuchten rot.
- Server aktivieren: Per Taste E wird ein Server hochgefahren, die LEDs wechseln auf grün.
- Stromausfall: Sobald alle Server der Runde aktiv sind, schließt sich die Tür, die Lampen flackern, an den Servern sprühen Funken und Spinnen fallen von der Decke.
- Zurück zum Schalter: Der Spieler kämpft sich zurück zum Power-Reset-Schalter vor der Tür. Beim Umlegen wird eine Lampe über der Tür grün, die alten Server verschwinden, neue erscheinen.

Runde 1 verlangt 3 Server, Runde 2 sechs und Runde 3 neun. Die Spinnen werden mit jeder Runde zahlreicher und stärker. In Runde 3 ist der Schalter nach dem Stromausfall für eine je nach Schwierigkeit vordefinierte Zeit gesperrt – ein Countdown läuft und eine Ansage erklärt die Sperre.

Diese Zeit muss man überleben. Danach aktiviert der Schalter das Selbstverteidigungssystem des Gebäudes: Alle Bugs verschwinden, die Tür öffnet sich, und wenn man den Starraum betritt, hat man gewonnen. Fallen die Lebenspunkte vorher auf null, ist das Spiel verloren.

4 Spielmechaniken

- Steuerung: WASD, Umschalt zum Sprinten, Maus zum Umsehen, E zum Interagieren, Escape für Pause.
- Interaktion: Ein Raycast aus der Kamera prüft, ob ein benutzbares Objekt in Reichweite ist. Wenn ja, erscheint „Interact (E)“. Startknopf, Server und Schalter laufen alle über dieselbe Schnittstelle. Ein bereits aktiver Server meldet sich selbst als „nicht mehr benutzbar“.
- Kampf: Eine Pistole, jeder Klick ein Schuss, keine Munition, kein Nachladen. Der Schuss ist ein sichtbares Projektil, das beim ersten Treffer Schaden macht und verschwindet. Der Kampf soll den Rückweg gefährlich machen, nicht das eigentliche Spiel sein.
- Lebenspunkte: 100 zu Beginn, jeder Spinnenbiss zieht Schaden ab, keine Regeneration. Ein Balken im HUD zeigt den Stand.
- Schalter und Lampenanzeige: Der Hebel hängt außen an der Wand des Starraums und ist nur während eines Stromausfalls benutzbar. Die drei Lampen über der Tür sind der Fortschrittsbalken des Spiels.

5 Gegner-KI

Die Logik der Gegner-KI ist in einer simplen FSM mit vier Zuständen zusammengefasst: Fall (von der Decke bis zum Boden), Patrol (zufällige Punkte in der Nähe ablaufen, dabei lose Gruppen mit anderen Spinnen bilden), Chase (Spieler über das NavMesh verfolgen) und Attack (in Nahkampfreichweite in festen Abständen zubeißen).

Jede Runde hat eine eigene Schwierigkeitsstufe, die Lebenspunkte, Schaden, Geschwindigkeit, Größe, maximale Anzahl und Spawn-Intervall der Spinnen festlegt. Die Werte steigen von Runde zu Runde.

Daneben krabbeln in jedem Raum kleine Käfer über den Boden. Sie greifen nicht an und sind reine Atmosphäre, haben aber ein echtes Schwarmverhalten: Abstand halten, in ähnliche Richtungen laufen, im Raum bleiben, dem Spieler ausweichen. Im ganzen Dungeon können bis zu tausend Käfer, deren Bewegung auf mehreren CPU-Kernen parallel berechnet wird (Threadoptimierung, siehe TDD).

6 Level-Design

Der Startraum ist der einzige handgebaute Raum und gleichzeitig der Endraum. Er enthält Startknopf, Energiefeld-Tür, Lampenanzeige und den Sieg-Trigger. Der Power-Reset-Schalter hängt außen.

Der Dungeon ist prozedural vorgeneriert aber der Spieler bekommt eine Version davon zu sehen. Grundlage sind 11 Raum-Prefabs, die sich in ihren Ausgängen unterscheiden. Jeder Ausgang hat ein Connector-Punkt. Der Generator nimmt einen offenen Connector, sucht ein Prefab mit passendem Gegenstück, setzt es exakt an und prüft auf Überlappung. Das läuft, bis die Maximalzahl an Räumen erreicht ist. Offene Ausgänge werden zum Schluss mit Wänden verschlossen, danach wird das NavMesh gebacken. Das ganze lässt sich auch mit einem Seed generieren um das gewünschte Dungeon fürs eigentliche Spiel auszusuchen.

Jedes Raum-Prefab bringt seine Spawnpunkte mit: für Server, für die Spinnen, für die Boden-Käfer und für Pflanzen, die mit einer gewissen Wahrscheinlichkeit zufällig gesetzt werden.

Für die Arbeit am Level gibt es ein Editor-Fenster (Tools → Dungeon Generator), in dem man Seed und Raumanzahl einträgt und den Dungeon direkt in der Szene aufbaut oder löscht, ohne das Spiel zu starten.

7 Audio und UI

- Shader: Monitor-Rauschen im Hauptmenü, Metallboden mit einstellbarem Glanz, Energiefeld-Tür mit Fresnel-Effekt.
- Partikel: Staub in der Luft, Mündungsblitz an der Waffe, Funken an den Servern beim Stromausfall.
- Licht: Die Deckenlampen sind das wichtigste Rückmeldesystem – ruhig im Normalzustand, unregelmäßiges Flackern beim Stromausfall.
- Audio: Menümusik, Ambience in der Spielszene, Schussgeräusch, eine Ansage beim Spielstart, die das Prinzip erklärt, und eine zweite in Runde 3 zum Timer.
- Hauptmenü: Start, How to Play, Options (Lautstärke), Credits, Quit.
- HUD: Lebensbalken, Interact-Hinweis, Timer in Runde 3. Keine Minimap, keine Zielmarkierung – das Suchen ist Teil des Spiels.

8 Änderungen im Laufe der Zeit

- Bug-Spray, Lockmittel, Bugbomben: Ersetzt durch eine Pistole. Drei Waffen hätten drei Modelle, Animationen und eine Art Inventar gebraucht. Das Spray lebt als Idee im Selbstverteidigungssystem weiter.
- Quick Time Events: Gestrichen. Ein QTE mitten in einem Raum voller Spinnen frustriert mehr, als das es Spaß macht.
- Prozedurale Möbelplatzierung: Reduziert auf Server und Pflanzen an festen Stellen aus Zeitgründen.
- Bosskampf „Bug Mutter“: Verworfen. Der Timer in Runde 3 übernimmt die Rolle der finalen Herausforderung.
- Getrennter Endraum: Start- und Endraum sind jetzt derselbe Raum aus Gründen zur Vereinfachung der Logik.

9 Verwendete Assets

- Spider – Low Poly – Animated von RetroVandal (itch.io), CC BY 4.0 – Namensnennung im Credits-Panel.
- Free Sci-Fi Office Pack von Terresquall (Unity Asset Store) – Wände, Böden, Server.
- Retro Weapon Pack (kostenlos, keine Namensnennung nötig) – die Pistole.
- Gogo Casual Free Plants Pack (Unity Asset Store) - die Pflanzen.

10 Offene Punkte

Balance der Schwierigkeitsstufen im Test feinjustieren

Audio fertigstellen

Credits-Panel mit allen Assets befüllen